13-understanding-test-doubles/exercises/mountain/spec/skier_spec.rb

```
require 'skier'

module Mountain
  RSpec.describe Skier do
    it 'gets tired after skiing a difficult slope' do
      trail_map = instance_double('TrailMap', difficulty: :expert)

      skier = Skier.new(trail_map)
      skier.ski_on('Last Hoot')
      expect(skier).to be_tired
    end
  end
end
```

This spec makes the same mistake the Skier class did with method names. It stubs the difficulty method instead of difficulty_of. However, you're using instance_double, so RSpec should catch the problem—right?

Try running your spec:

```
$ rspec
```

Surprisingly, the specs pass. RSpec can only verify against a real class if that class is actually loaded. With nothing to verify against, the verifying double acts just like a normal, non-verifying double. So, try running it again *with* the TrailMap class loaded; just pass -rtrail_map on the command line:

```
$ rspec -rtrail_map
```

The specs *still* pass. Moreover, they're running much more slowly (nearly 10x slower on our computers!) because of the time spent loading a heavyweight dependency. Before moving on, see if you can guess why RSpec isn't checking your trail_map double against the real Mountain::TrailMap class.

The Problem

The problem is that the constant name passed into instance_double doesn't match the real class. The TrailMap class's full name, including the module it's nested in, is 'Mountain::TrailMap'.

Change the instance_double call to use the correct name, and then rerun your specs (again, with -rtrail_map). This time, they should fail the way you'd expect them to: with an error message about the use of a nonexistent difficulty method.

There are two ways to catch these kinds of naming issues before they happen:

- Use Ruby classes instead of strings
- Configure RSpec to check that the class name exists